



UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

Department of Internet of Things and Robotics Engineering

ASSIGNMENT — 01 Solution

IRE 313 – Modeling and Simulation of Robotic System

SUBMITTED TO

Mostafiz Ahammed

Lecturer

Department of Internet of Things and Robotics Engineering

University of Frontier Technology, Bangladesh

SUBMITTED BY

Sl.	Class Roll	Name	Session
01	2201022	Mst. Esat Jahan Akhi	2022–2023

DATE OF SUBMISSION

September 2, 2026

Answer to the question No. 1

A custom hardware interface class in ROS controll is created by inheriting from `hardware_interface::RobotHW`. This class acts as the abstraction layer connecting ROS controllers to the actual robot hardware (motors & sensors).

```
#include <hardware_interface/robot_hw.h>

class MyRobot :
    public hardware_interface::RobotHW
{
public:
    MyRobot();
    MyRobot(ros::NodeHandle &nh, urdf::Model *urdf_model
            = NULL);

    // Talk to HW
    void read();
    void write();

    // Requirement only if needed
    virtual bool checkForConflict(...) const;
}
```

Explanation of each part :

1. `#include <hardware_interface/robot_hw.h>`

This includes the header file for `hardware_interface` :: `RobotHW`, the base class provided by `ros_control` that must be inherited to define a robot's hardware interface. Without this include, the compiler would not know what `hardware_interface` :: `RobotHW` is.

2. `class MyRobot : public hardware_interface :: RobotHW`

This declares a new C++ class named `MyRobot`. The `:` indicates inheritance, and `public hardware_interface :: RobotHW` means `MyRobot` inherits publicly from `RobotHW`. So `MyRobot` is a specialized `RobotHW` that must implement how to read from and write to the real (or simulated) robot.

3. `public :`

This is an access specifier. Everything after `public :` is accessible from outside the class (e.g. other code can call `MyRobot :: read()`).

4. `MyRobot();` // setup robot.

This is default constructor declaration. It is responsible for initializing internal variables, setting up communication (e.g. CAN, serial, etc) and preparing the robot for use.

5. `MyRobot (ros::NodeHandle &nh, urdf::Model * urdf_model = NULL);`

This is an overloaded constructor declaration. It is responsible for ~~initializing internal variables, setting up communication~~ (e.g. CAN, serial, etc) and preparing the robot for use. It loads the robot configuration from the parameter server, reads the URDF, and registers joints and interfaces with RobotHW.

6. `void read();`

This function reads the current state from the hardware, including joint positions, joint velocities, and joint efforts.

7. `void write();`

This function writes commands from the ROS controllers to the hardware, including position commands, velocity commands and effort/torque commands.

8. virtual bool checkForConflict(...) const;

This function is used when loading or switching controllers. It checks whether two controllers are conflicting (e.g. both trying to control the same joint in incompatible ways). It returns true if there is a conflict and false if it is safe ~~on~~ to proceed.

Answer to the question No. 2

To create and integrate a simple RViz panel plugin, follow these three required components:

1. Python Plugin file (button_panel.py)

This file defines the RViz panel. It creates a button and logs a message when clicked.

```
from python_qt_binding.QtWidgets import QPushButton, QVBoxLayout
```

```
import rviz
```

```
import rospy
```

```
class ButtonPanel (rviz.Panel):
```

```
    def __init__(self, parent = None):
```

```
        super(ButtonPanel, self).__init__(parent)
```

```
        layout = QVBoxLayout()
```

```
        self.button = QPushButton("click me")
```

```
        self.button.clicked.connect(self.on_click)
```

```
        layout.addWidget(self.button)
```

```
        self.setLayout(layout)
```

```
    def on_click(self):
```

```
        rospy.loginfo("Button was clicked!")
```

2. Plugin Description XML (button-panel.xml)

This file tells RViz how to load the python class as a plugin.

```
<class_library path = "lib/libbutton-panel">
  <class
    name = "button-panel/ButtonPanel"
    type = "button-panel, ButtonPanel"
    base_class_type = "rviz::Panel">
    <description> Simple RViz panel with one button</description>
  </class>
</class_library>
```

3. CMakeLists.txt. Additions

This installs the python script and XML file.

```
catkin_install_python(
  PROGRAMS src/button-panel.py
  DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
)
install (FILES button-panel.xml
  DESTINATION ${CATKIN_PACKAGE_SHARE_DESTINATION}
)
```

How to Build and Run the Plugin :

1. Build the workspace

`catkin-make`

`source devel/setup.bash`

2. Launch RViz

`rviz`

3. Add your custom panel, inside RViz :

Panel → Add New Panel → `button-panel/ ButtonPanel`

clicking the button prints to your terminal :

[INFO] Button was clicked !